Engineering Scope Definition

Project Name

CarbonWise

Version: 1.0

Status: Approved for Development

---

1. Purpose

This document defines the engineering boundaries, implementation scope, technical responsibilities, and system constraints for CarbonWise.

The objective is to ensure:

- Clear development direction
- Maintainable architecture
- High code quality
- Secure implementation
- Efficient system performance
- Alignment with challenge requirements

---

2. Engineering Goals

Primary Goals

Build a system that helps users:

- Understand their carbon footprint
- Track emissions over time
- Receive personalized sustainability insights
- Reduce emissions through actionable recommendations

Technical Goals

- Maintain clean architecture
- Ensure scalability

- Support future feature expansion
- Achieve high testability
- Maximize maintainability

---

3. Technology Stack

Frontend

Framework

```
Next.js 15
```

Language

```
TypeScript
```

Styling

```
Tailwind CSS
```

Component System

```
Shadcn/UI
```

State Management

```
TanStack Query
```

Backend

Framework

```
NestJS
```

Reason:

- Strong architecture
- Dependency Injection
- Testability
- Enterprise-grade structure

Language

```
TypeScript
```

Database

Primary Database

```
PostgreSQL
```

ORM

```
Prisma ORM
```

Benefits:

- Type Safety
- Schema Management
- Migration Support

Cache Layer

Technology

```
Redis
```

Used for:

- Dashboard Caching
- Session Management
- Recommendation Caching

Authentication

Method

```
JWT Authentication
```

Password Security

```
bcrypt
```

Testing

Unit Testing

```
Jest
```

Integration Testing

```
Supertest
```

End-to-End Testing

```
Playwright
```

---

4. Project Scope

Included Features

User Authentication

Capabilities:

- Registration
- Login
- Logout
- Password Reset
- Session Management

Carbon Calculator

Categories:

Transportation

- Car
- Bus
- Train
- Flight
- Bike

Food

- Meat
- Dairy
- Vegetarian
- Vegan

Energy

- Electricity

- Heating
- Cooling

Shopping

- Clothing
- Electronics
- Household Goods

Dashboard

Features:

- Carbon Score
- Emission Trends
- Category Breakdown
- Sustainability Rating

Recommendation Engine

Features:

- Personalized Recommendations
- Priority Ranking
- Impact Estimation
- Action Plans

Goal Tracking

Features:

- Create Goals
- Track Progress
- Milestones
- Goal Completion

Carbon Twin

Features:

- What-if Simulations

- Future Emission Forecasts
- Sustainability Planning

---

5. Excluded Scope

The following features are intentionally excluded.

Carbon Credit Trading

Reason:

Outside challenge requirements.

Government Integrations

Reason:

Requires external dependencies.

Smart Home Device Integration

Reason:

Requires hardware infrastructure.

Enterprise ESG Reporting

Reason:

Focus remains on individual users.

Marketplace Features

Reason:

Not directly aligned with problem statement.

---

6. Frontend Scope

Responsibilities

Frontend will handle:

- User Interface
- User Experience
- Data Visualization
- Form Handling
- Client-side Validation

Pages

Authentication

- Login
- Register
- Forgot Password

Dashboard

- Carbon Overview
- Trends
- Recommendations

Calculator

- Transportation
- Food
- Energy
- Shopping

Goals

- Active Goals
- Completed Goals

Carbon Twin

- Scenario Simulation
- Forecast Visualization

Profile

- Preferences
- Settings

---

7. Backend Scope

Responsibilities

Backend will handle:

- Business Logic
- Authentication
- Recommendation Logic
- Carbon Calculations
- Forecasting

Services

Auth Service

Handles:

- Login
- Registration
- JWT Generation

User Service

Handles:

- Profiles
- Preferences
- Settings

Carbon Service

Handles:

- Carbon Calculations
- Emission Factors
- Carbon Records

Recommendation Service

Handles:

- User Analysis
- Recommendation Generation
- Recommendation Ranking

Analytics Service

Handles:

- Dashboard Metrics
- Trends
- Reports

Carbon Twin Service

Handles:

- Simulations
- Future Forecasting

---

8. Database Scope

User Data

Stores:

- Profiles
- Preferences
- Goals

Activity Data

Stores:

- Transportation Records
- Food Records
- Energy Records
- Shopping Records

Recommendation Data

Stores:

- Recommendations
- Acceptance Status
- Impact Estimates

Analytics Data

Stores:

- Carbon Scores
- Historical Trends
- Monthly Reports

---

9. Security Scope

Helmet

Used for:

- Security Headers
- Clickjacking Prevention
- MIME Sniffing Protection

Rate Limiting

Purpose:

Prevent API abuse.

Input Validation

Technology:

```
Zod
```

Purpose:

Prevent malformed requests.

Authentication

JWT-based access control.

Password Security

bcrypt hashing.

Environment Security

Secrets stored in:

```
.env
```

Excluded from repository.

SQL Injection Protection

Provided through:

```
Prisma ORM
```

---

10. Efficiency Scope

Database Indexing

Indexes:

- user_id
- email
- created_at

Caching

Redis caches:

- Dashboard Data
- Recommendations
- Frequently Accessed Queries

Pagination

Implemented on:

- Activity History
- Recommendation Lists

Lazy Loading

Applied to:

- Charts
- Analytics
- Historical Data

---

11. Testing Scope

Unit Tests

Coverage Targets:

```
90%
```

Modules:

- Carbon Calculations
- Recommendation Engine
- Goal Logic

Integration Tests

Coverage Targets:

```
80%
```

Areas:

- API Endpoints
- Database Operations
- Authentication

End-to-End Tests

Technology:

```
Playwright
```

Flows:

- Registration
- Login
- Carbon Calculation
- Recommendation Generation
- Goal Tracking

---

12. Accessibility Scope

WCAG 2.2 AA

Required Compliance.

Keyboard Navigation

Supported.

Screen Readers

Supported.

Color Contrast

Minimum:

```
4.5:1
```

Responsive Design

Supported on:

- Desktop
- Tablet
- Mobile

---

13. Code Quality Standards

Architecture

Follow:

- SOLID Principles
- Clean Architecture

- Separation of Concerns

Naming

Use:

- Consistent Naming Conventions
- Self-Documenting Code

Documentation

Required:

- README
- API Documentation
- Inline Comments for Complex Logic

Linting

Tool:

```
ESLint
```

Formatting

Tool:

```
Prettier
```

---

14. Development Constraints

Repository Requirements:

- Public Repository
- Single Branch

- Under 10 MB

Development Requirements:

- TypeScript Only
- Git Version Control
- Automated Testing

---

15. Deliverables

Frontend

- Next.js Application

Backend

- NestJS API

Database

- PostgreSQL Schema

Testing

- Jest Tests
- Supertest Tests
- Playwright Tests

Documentation

- README
- Architecture Docs
- API Docs

---

16. Success Criteria

Engineering is considered successful when:

✅ Carbon calculations are accurate

✅ Recommendations are personalized

✅ Security best practices are implemented

✅ Performance remains efficient

✅ Accessibility requirements are satisfied

✅ Codebase remains maintainable

✅ Test coverage targets are achieved

✅ Users can understand, track, and reduce their carbon footprint effectively